Solve Single Number III XOR

Yes. This is a classic **XOR + bit manipulation** problem. The key is to use the fact that XOR cancels numbers appearing twice.

1. First understand XOR

The important properties are:

```
a ^ a = 0
a ^ 0 = a
a ^ b = b ^ a
```

So if we have:

```
[1, 2, 1, 3, 2, 5]
```

XOR everything:

```
1 ^ 2 ^ 1 ^ 3 ^ 2 ^ 5
```

Rearrange:

```
(1 ^ 1) ^ (2 ^ 2) ^ 3 ^ 5
```

Therefore:

```
0 ^ 0 ^ 3 ^ 5
= 3 ^ 5
```

So after one XOR pass, we don't get **3** and **5** separately.

We get:

```
xorAll = 3 ^ 5
```

2. How do we separate 3 and 5?

This is the important bit-manipulation trick.

```
3 = 011
5 = 101
```

XOR:

```
  011
^ 101
-----
  110
```

So:

```
3 ^ 5 = 6
```

Look at **6**:

```
6 = 0110
```

There is at least **one bit where 3 and 5 are different**.

For example, the rightmost set bit of **6** is:

```
6 = 0110
       ↑
```

That bit allows us to divide all numbers into **two groups**:

```
Group 1 → numbers having this bit = 0
Group 2 → numbers having this bit = 1
```

Because **3** and **5** differ at this bit, they will go into **different groups**.

But every duplicate pair is identical, so both copies always go into the **same group** and cancel each other.

That's the entire trick.

3. Finding the rightmost set bit

Suppose:

```
xorAll = 3 ^ 5 = 6
```

Binary:

```
6 = 0110
```

We want the rightmost **1**.

There is a very useful trick:

```
C++
xorAll & (-xorAll)
```

For **6**:

```
  0110
& 1010    (-6 in two's complement)
------
  0010
```

So:

```
rightmostBit = 2
```

4. Now divide the numbers into two groups

Our array:

```
[1, 2, 1, 3, 2, 5]
```

And:

```
rightmostBit = 2
```

Check:

```
num & rightmostBit
```

Number 1

```
1 = 0001
2 = 0010

0001 & 0010 = 0000
```

Group 0.

Number 2

```
0010 & 0010 = 0010
```

Group 1.

Number 1

Group 0 again.

Number 3

```
0011 & 0010 = 0010
```

Group 1.

Number 2

Group 1 again.

Number 5

```
0101 & 0010 = 0000
```

Group 0.

So:

```
             rightmostBit = 2
                    |
                    ↓

Group 0                 Group 1
--------                --------
1                       2
1                       3
5                       2
```

Now XOR each group.

Group 0

```
1 ^ 1 ^ 5
= 0 ^ 5
= 5
```

Group 1

```
2 ^ 3 ^ 2
= 0 ^ 3
= 3
```

Therefore:

```
answer = [5, 3]
```

5. Complete algorithm

There are only **two passes**.

Pass 1 — XOR everything

```
xorAll = a ^ b
```

where **a** and **b** are the two unique numbers.

Pass 2 — Find a distinguishing bit

```
C++
int bit = xorAll & (-xorAll);
```

This gives a bit where **a** and **b** are different.

Pass 3 — Split and XOR

For every number:

```
C++
if (num & bit)
    x ^= num;
else
    y ^= num;
```

Duplicates cancel inside their respective groups.

C++ Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        int xorAll = 0;

        // Step 1: XOR all numbers
        for (int num : nums) {
            xorAll ^= num;
        }

        // Step 2: Find rightmost set bit
        int bit = xorAll & (-xorAll);

        int x = 0;
        int y = 0;
```

```
        // Step 3: Divide numbers into two groups
        for (int num : nums) {

            if (num & bit)
                x ^= num;
            else
                y ^= num;
        }

        return {x, y};
    }
};
```

6. The whole idea in one diagram

```
             nums
              |
              ↓
     [1, 2, 1, 3, 2, 5]
              |
              | XOR everything
              ↓
         1^2^1^3^2^5
              |
              ↓
            3 ^ 5
              |
              ↓
              6
         0110 (binary)
              |
              | find rightmost 1
              ↓
              2
         0010 (binary)
              |
              ↓
        ┌─────┴─────┐
        |           |
 bit = 0           bit = 1
        |           |
        ↓           ↓
    [1,1,5]       [2,3,2]
        |           |
        | XOR       | XOR
        ↓           ↓
        5           3
        |           |
        └─────┬─────┘
              |
              ↓
           [5, 3]
```

Why does this always work?

The crucial observation is:

```
unique1 ^ unique2 != 0
```

Therefore, they **must differ in at least one bit**.

We select one such bit.

That bit separates the two unique numbers:

```
unique1 → Group A
unique2 → Group B
```

Every duplicate pair consists of identical numbers:

```
x → Group A
x → Group A
```

or

```
x → Group B
x → Group B
```

Therefore:

```
x ^ x = 0
```

and only the two unique numbers survive.

Complexity:

```
Time  = O(n)
Space = O(1)
```

This is why this solution satisfies the problem's requirement of **linear time + constant extra space**.

Yes. For **LeetCode 260 — Single Number III**, there are several approaches. Some are straightforward but violate the `O(1)` extra-space requirement, while the XOR approach satisfies all constraints.

I'll show you the main approaches from simplest → optimal.

---

1. Brute Force — Nested Loops

For every number, count how many times it occurs.

If its count is `1`, add it to the answer.

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        vector<int> ans;

        for (int i = 0; i < nums.size(); i++) {

            int count = 0;

            for (int j = 0; j < nums.size(); j++) {
                if (nums[i] == nums[j]) {
                    count++;
                }
            }

            if (count == 1) {
                ans.push_back(nums[i]);
            }
        }

        return ans;
    }
};
```

Complexity

```
Time  = O(n²)
Space = O(1)   // excluding answer
```

This works, but is too slow for large `n`.

---

2. Sorting

Another simple idea:

Sort the array.

After sorting, duplicate numbers become adjacent.

For:

```
[1, 2, 1, 3, 2, 5]
```

After sorting:

```
[1, 1, 2, 2, 3, 5]
```

Now check pairs:

```
1 1 → duplicate
2 2 → duplicate
3   → unique
5   → unique
```

Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        sort(nums.begin(), nums.end());

        vector<int> ans;

        int n = nums.size();

        for (int i = 0; i < n; ) {

            if (i + 1 < n && nums[i] == nums[i + 1]) {
                i += 2;
            }
            else {
```

```
                ans.push_back(nums[i]);
                i++;
            }
        }

        return ans;
    }
};
```

Complexity

```
Time  = O(n log n)
Space = O(1)
```

Depending on the sorting implementation, auxiliary stack space can be **`O(log n)`**.

Important

This satisfies the **time/space reasonably well**, but it doesn't satisfy the required **`O(n)`** runtime.

---

3. Hash Map / Frequency Map

This is probably the easiest approach to understand.

Store:

```
number → frequency
```

For:

```
[1,2,1,3,2,5]
```

we get:

```
1 → 2
2 → 2
3 → 1
5 → 1
```

Then return numbers whose frequency is **`1`**.

Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        unordered_map<int, int> freq;

        for (int num : nums) {
            freq[num]++;
        }

        vector<int> ans;

        for (auto &[num, count] : freq) {

            if (count == 1) {
                ans.push_back(num);
            }
        }

        return ans;
    }
};
```

Complexity

Average:

```
Time  = O(n)
Space = O(n)
```

This is very commonly used in interviews when the constant-space requirement isn't present.

---

4. Using `set`

We can exploit the fact that duplicates should disappear.

For each number:

```
if number already exists:
    remove it
else:
    insert it
```

At the end, only the two unique numbers remain.

Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        unordered_set<int> st;

        for (int num : nums) {

            if (st.count(num)) {
                st.erase(num);
            }
            else {
                st.insert(num);
            }
        }

        vector<int> ans(st.begin(), st.end());

        return ans;
    }
};
```

For:

```
[1,2,1,3,2,5]
```

Processing:

```
1 → {1}
2 → {1,2}
1 → {2}
3 → {2,3}
2 → {3}
5 → {3,5}
```

Final:

```
{3,5}
```

Complexity

Average:

```
Time  = O(n)
Space = O(n)
```

5. XOR — The Optimal Approach

This is the important one because it satisfies **both constraints**:

```
Time  = O(n)
Space = O(1)
```

First XOR everything:

```
C++
int xorAll = 0;

for (int num : nums) {
    xorAll ^= num;
}
```

All duplicates disappear:

```
1 ^ 2 ^ 1 ^ 3 ^ 2 ^ 5

= (1^1) ^ (2^2) ^ 3 ^ 5

= 3 ^ 5
```

Then find a bit where **3** and **5** differ:

```
C++
int bit = xorAll & (-xorAll);
```

Then divide numbers into two groups based on that bit.

Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        int xorAll = 0;

        // XOR of all numbers
        for (int num : nums) {
            xorAll ^= num;
        }

        // Rightmost set bit
        int bit = xorAll & (-xorAll);

        int a = 0;
        int b = 0;

        // Divide into two groups
        for (int num : nums) {

            if (num & bit) {
                a ^= num;
            }
            else {
                b ^= num;
            }
        }

        return {a, b};
    }
};
```

6. Another XOR Variant — Using `x & ~(x-1)`

The rightmost set bit can also be obtained using:

```
C++
int bit = xorAll & ~(xorAll - 1);
```

instead of:

```
C++
int bit = xorAll & (-xorAll);
```

Both do the same thing.

Code

```
C++
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {

        int xorAll = 0;

        for (int num : nums) {
            xorAll ^= num;
        }

        int bit = xorAll & ~(xorAll - 1);

        int a = 0;
        int b = 0;

        for (int num : nums) {

            if (num & bit)
                a ^= num;
            else
                b ^= num;
        }

        return {a, b};
    }
};
```

Comparison of all approaches

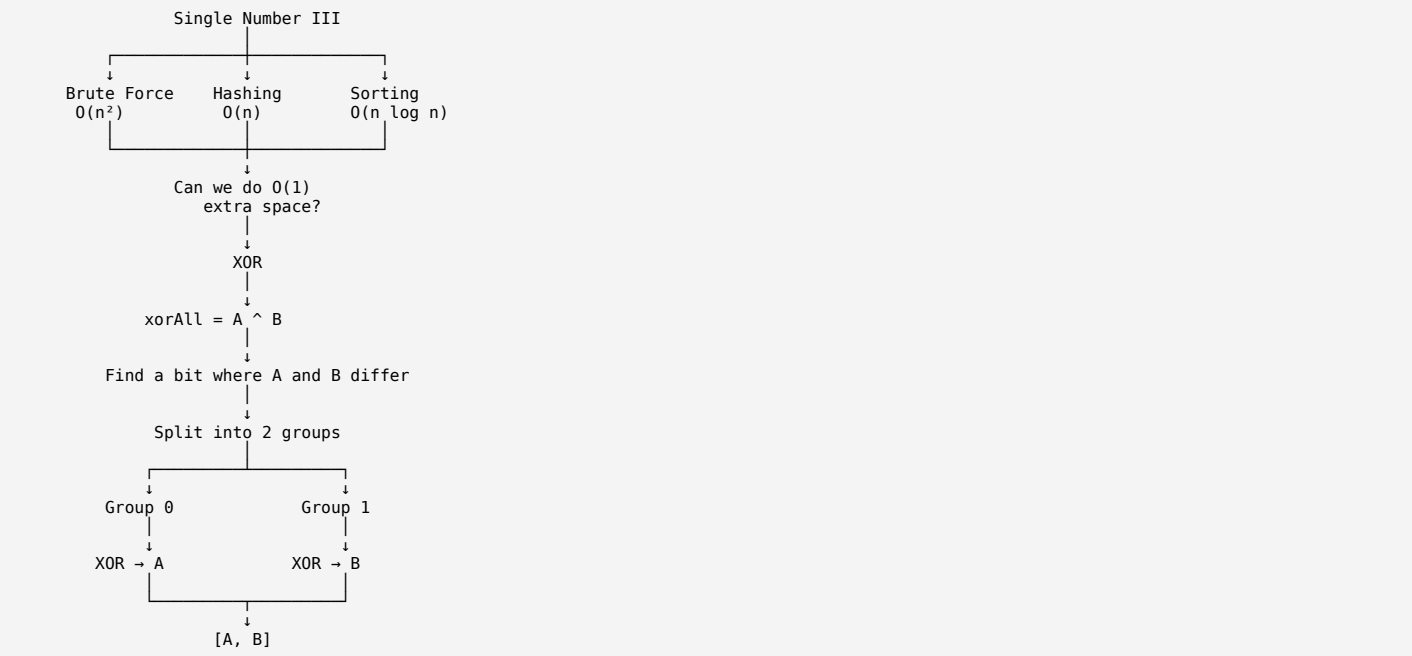
Approach	Time	Extra Space	Meets Optimal Requirement?
Brute Force	$O(n^2)$	$O(1)$	❌
Sorting	$O(n \log n)$	$O(1)$*	❌
Hash Map	$O(n)$	$O(n)$	❌

Approach	Time	Extra Space	Meets Optimal Requirement?
Hash Set	**O(n)** average	**O(n)**	❌
XOR + Bit Manipulation	**O(n)**	**O(1)**	✅

*Ignoring the sorting algorithm's internal stack space.

The progression you should remember

Think about the problem like this:



For interviews, know all four basic ideas, but the one you should actually use for this problem is the last one: XOR + rightmost set bit.